Monte Carlo study of the Muon-Decay electron spectrum with radiative corrections

Estiven Castrillon Alzate
Instituto de Física, Universidad de Antioquia
estiven.castrillon2@udea.edu.co

***Abstract*—This report documents a Python Monte Carlo study of the electron energy spectrum in the decay $\mu^- \to e^- \bar{\nu}_e \nu_\mu$, evaluating the Michel spectrum and an α-order radiative correction. For each correction setting, the analysis driver performs 100 runs of 100,000 accepted events, concatenates their energies, and makes one histogram from the resulting 10-million-event sample. The calculation is motivated by the spectrum discussed by Czarnecki et al. for a muon bound to a nucleus, however, the implemented formula is the free-muon spectrum and does not include atomic binding or nuclear-structure effects. The driver produces separate histograms for the two correction settings. The sampler still estimates its envelope on a finite grid and samples the vertical coordinate from the estimated minimum rather than from zero, so repeating the runs does not remove this per-event sampling limitation. The report explains the physical model, updated run procedure, and scope of the generated plots.**

***Index Terms*—Muon decay, Michel spectrum, Monte Carlo, rejection sampling, radiative corrections.**

I. Introduction

Muon decay is a process for studying weak interactions and precision quantum electrodynamics (QED), for a muon at rest, the main decay mode is

$$\mu^- \longrightarrow e^- + \bar{\nu}_e + \nu_\mu. \tag{1}$$

The electron energy is continuous because the two neutrinos carry away a variable share of the available energy, its spectrum is commonly expressed in terms of a dimensionless energy variable, making the shape easy to compare in calculations.

This repository combines that spectrum with the Monte Carlo acceptance-rejection method, it evaluates a spectrum and radiative correction, draws electron energies for each choice, saves samples and figures, and provides a notebook for comparing the distributions.

The principal physics reference is Czarnecki et al., who study the Michel spectrum for a muon bound to a nucleus [1]. The paper motivates the radiative-correction formula used here, but the implementation in this repository uses the free-muon expression, a bound muon experiences the nuclear Coulomb field, and its spectrum requires bound state physics not present in this program due to its complexity. The work is therefore a free-muon spectrum exercise inspired by the paper, not a simulation of the full bound-muon calculation or a reproduction of the paper's experimental comparison.

II. Physics model

A. Normalized energy and kinematic range

The code uses the normalized electron energy

$$x = \frac{2E_e}{m_\mu}, \qquad E_e = \frac{x m_\mu}{2}, \tag{2}$$

where E_e and m_μ are expressed in the same energy units.

B. Main Michel spectrum

At tree level, neglecting terms suppressed by m_e^2/m_μ^2, the differential decay rate is

$$\frac{d\Gamma_0}{dx} = C\, x^2(6-4x), \qquad C = \frac{G_F^2 m_\mu^5}{192\pi^3}, \tag{3}$$

where G_F is the Fermi coupling constant. The shape factor $x^2(6-4x)$ is nonnegative on the physical interval and vanishes at the low energy endpoint, the overall constant C sets the rate scale where the kinematic range is $0 < x \leq 1$, corresponding to $0 < E_e \leq m_\mu/2$.

C. Order-α radiative correction

The implementation adds the order-α term to the main shape:

$$\frac{d\Gamma}{dx} = C\, x^2 \left[6 - 4x + \frac{\alpha}{\pi} f(x)\right], \tag{4}$$

where α is the fine-structure constant. Terms beyond first order in α are omitted. The correction used in the program is

$$\begin{aligned} f(x) = {} & \left[\frac{5}{3x^2} + \frac{16x}{3} + \frac{4}{x} \right. \\ & \left. \quad +(12-8x)\ln\!\left(\frac{1}{x}-1\right) - 8\right] \ln\!\left(\frac{m_\mu}{m_e}\right) \\ & + (6-4x)\left[2\,\mathrm{Li}_2(x) - 2\ln^2 x + \ln x \right. \\ & \left. \quad + \ln(1-x)\left(3\ln x - \frac{1}{x} - 1\right) - \frac{\pi^2}{3} - 2\right] \\ & + \frac{1-x}{3x^2}\left[34x^2 + (5 - 34x^2 + 17x)\ln x - 22x\right] \\ & + 6(1-x)\ln x. \end{aligned} \tag{5}$$

Here m_e is the electron mass and $\mathrm{Li}_2(x)$ is the dilogarithm, sciPy's Spence function obeys $\mathrm{spence}(z) = \mathrm{Li}_2(1-z)$, so the code's call `spence(1 - x)` evaluates $\mathrm{Li}_2(x)$. The logarithms and inverse powers make the correction numerically sensitive near $x = 0$ and $x = 1$.

The constants are obtained from the `particle` package and `scipy.constants.physical_constants`, the latter follows CODATA/NIST physical-constant data [2]. There is an important unit caveat for absolute rates: the Fermi constant supplied by SciPy is conventionally tabulated in GeV^{-2}, whereas the particle masses in this project are in MeV. The code combines these values without an explicit conversion, but since C is a common positive factor that cancels from the ideal normalized sampling distribution, this does not change the intended shape of the spectrum compared to the paper.

III. Monte Carlo method

A. Acceptance-rejection derivation

For a proposal density $q(x)$ and a constant M satisfying $p(x) \le Mq(x)$ throughout the sampling interval, the standard procedure is:

1) Draw a candidate $X \sim q$.
2) Draw $U \sim \mathcal{U}(0,1)$ independently.
3) Accept X when $U \le p(X)/(Mq(X))$, otherwise reject it.

For the uniform proposal on $[x_{\min}, x_{\max}]$, $q(x) = 1/(x_{\max} - x_{\min})$. Equivalently, one may draw a point uniformly below an envelope $M_f \ge \max_x(d\Gamma/dx)$ and accept it when

$$Y \le \frac{d\Gamma}{dx}(X), \qquad X \sim \mathcal{U}(x_{\min}, x_{\max}), \quad Y \sim \mathcal{U}(0, M_f). \tag{6}$$

The accepted values then follow the target density, and the expected acceptance fraction is the area under the target divided by the area under the envelope, the method and its assumptions are discussed in standard Monte Carlo references [3]–[5].

B. Implementation in this repository

The class `muonMonteCarlo` in `src/python/muonMonteCarlo.py` evaluates $f(x)$ and $d\Gamma/dx$ with NumPy arrays. It masks values outside $0 < x \le 1$, uses a batch size of 500,000 candidates, and tracks the number of accepted events with `tqdm`. Accepted normalized energies are converted to MeV using Eq. (2). The analysis script `scripts/run_analysis.py` processes one case with radiative corrections and one without them. For each case it calls the simulation 100 times with 100,000 accepted events per call, concatenates the 100 energy arrays, and plots the combined 10-million-event sample. Both cases use

$$x_{\min} = 4.770585 \times 10^{-3}, \qquad x_{\max} = 0.99999. \tag{7}$$

The lower cutoff was chosen from a numerical scan because the order-α rate becomes negative at sufficiently small x, it is an empirical cutoff for this setup, not a universal physical threshold. The upper cutoff avoids evaluating the logarithmic endpoint terms at exactly $x = 1$. Each call to `simulation` saves a paired array using a filename based on the correction setting and per-run event count. Repetitions use the same filename, so the files contain only the final 100,000-event run, not the concatenated sample used in the plot. The driver does not set an explicit seed, so it does not record reproducible seeds for the individual runs.

The rejection routine estimates both the minimum and maximum rates on a finite grid and draws its vertical coordinate from that estimated minimum to maximum. If the minimum is nonzero, the accepted density is proportional to the rate minus that minimum rather than to the rate itself, a finite grid also does not establish a global maximum. Repeating and concatenating runs increases the sample count but does not correct this limitation or by itself validate the target distribution.

IV. Software and analysis workflow

The repository's main components are:

- `src/python/muonMonteCarlo.py`: constants, spectrum functions, sampling, data output, and histogram generation.
- `scripts/run_analysis.py`: the two configured simulation cases and plot labels.
- notebooks/01_analysis_of_results/muon_decay_rate_analysis.ipynb: spectrum comparison, histograms, and summary statistics.
- `tests/test.py`: basic checks of initialization, rate shape, sample count, range, saved data, and plot creation.
- `tests/testing_pdf_function.py`: exploratory boundary-limit and sign scans for the corrected rate.

V. Results

The current script produces one count histogram per correction setting from 100 runs of 100,000 accepted events each (10 million plotted events per setting). The individual current plots are shown below. The notebook and its embedded comparison are from an earlier 100-million-event analysis, its summary values are historical and are not measurements from the current 10-million-event plots:

TABLE I
Summary reported by the saved analysis-notebook output. Peak density is the height of the maximum histogram bin and depends on the binning.

Configuration	Peak density	Peak-bin center
No radiative term	3.7916×10^{-2}	51.5019
Order-α correction	3.6161×10^{-2}	50.2475

The historical values in the table are finite-bin histogram summaries, not fitted peak locations or uncertainties. The repeated-run driver prints the minimum and maximum energy of the concatenated sample. The console output supplied for the updated run reports a range of approximately 0.2657 MeV to 52.8287 MeV, this is an observed range, not a fitted endpoint or a peak measurement. Radiative corrections alter the shape of the spectrum, while the kinematic endpoint is unchanged.

The comparison figure and peak-bin table are retained from the earlier notebook analysis and refer to its 10^8-event input files, not the current driver output. The latest driver writes the

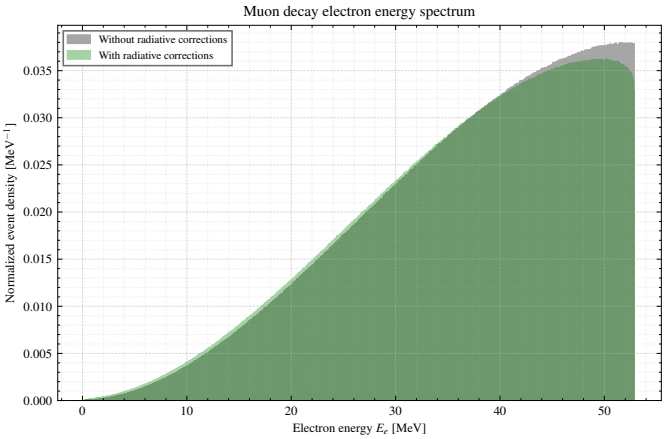


Fig. 1. Normalized electron-energy histograms with and without the radiative correction, as saved by the analysis notebook.

two current individual plots but saves only the last 100,000-event array for each case because each repetition overwrites the same filename. The combined samples used for the plots are not persisted. The project also contains a small test plot, which is not a physics result.

The current run script produces 10^7 accepted events per configuration from 100 runs of 10^5 events, in addition to rejected candidates, runtime and memory use can be substantial because the combined energy samples are held in memory for plotting, the generated values are conditional on the configured interval and on the current sampler implementation.

VI. Conclusion

This project is a useful worked example connecting a lepton-decay spectrum to Monte Carlo sampling, vectorized Python code, saved data, tests, and visual analysis. The reported comparison illustrates that adding the order-α term changes the spectrum's shape while leaving the energy endpoint fixed, physically precise interpretation.

References

[1] A. Czarnecki, M. Dowling, X. Garcia i Tormo, W. J. Marciano, and R. Szafron, "Michel decay spectrum for a muon bound to a nucleus," *Physical Review D*, vol. 90, no. 9, Nov. 2014. [Online]. Available: http://dx.doi.org/10.1103/PhysRevD.90.093002

[2] N. I. of Standards and Technology. (2022) Fundamental physical constants. Accessed: August 3, 2026. [Online]. Available: https://physics.nist.gov/cuu/Constants/

[3] S. Širca, *Probability for Physicists*, ser. Graduate Texts in Physics. Cham, Switzerland: Springer International Publishing, 2016, iSSN 1868-4513; ISSN 1868-4521 (electronic).

[4] W. L. Dunn and J. K. Shultis, *Exploring Monte Carlo Methods*. Amsterdam, The Netherlands: Elsevier Academic Press, 2012, library of Congress Control Number: 2010050137.

[5] A. Nakade. (2025, Feb.) Rejection sampling – monte carlo methods lecture notes. Monte Carlo Methods Lecture Notes. Accessed: 2026-04-28. [Online]. Available: https://apurvanakade.github.io/Monte-Carlo-Methods/chapters/sampling/accept_reject.html